

# **Values, Dynamic Typing and Desugaring**

CS 350

---

Dr. Joseph Eremondi

March 10, 2025

Introducing Boolean

Desugaring

# Introducing Boolean

---

- `if 0` was an awkward feature

- `if 0` was an awkward feature
  - Can't really do `and`, `or`

- `if 0` was an awkward feature
  - Can't really do `and`, `or`
  - e.g. What should `{and 1 1}` be?

- `if 0` was an awkward feature
  - Can't really do `and`, `or`
  - e.g. What should `{and 1 1}` be?
    - Any non-zero value has the right properties

- `if 0` was an awkward feature
  - Can't really do `and`, `or`
  - e.g. What should `{and 1 1}` be?
    - Any non-zero value has the right properties
- We want our interpreter to have multiple *types*



- `if 0` was an awkward feature
  - Can't really do `and`, `or`
  - e.g. What should `{and 1 1}` be?
    - Any non-zero value has the right properties
- We want our interpreter to have multiple *types*
  - Numbers and Booleans

## Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs

# Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs
  - E.g. Flit, C++, Go, Java, Swift, Rust, Haskell, TypeScript, etc.

## Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs
  - E.g. Flit, C++, Go, Java, Swift, Rust, Haskell, TypeScript, etc.
- Dynamic typing: types are checked at run-time

# Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs
  - E.g. Flit, C++, Go, Java, Swift, Rust, Haskell, TypeScript, etc.
- Dynamic typing: types are checked at run-time
  - Don't check if an operation is valid until we're actually doing that operation

# Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs
  - E.g. Flit, C++, Go, Java, Swift, Rust, Haskell, TypeScript, etc.
- Dynamic typing: types are checked at run-time
  - Don't check if an operation is valid until we're actually doing that operation
  - E.g. Python, Ruby, JavaScript, PHP, Racket, Lua

# Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs
  - E.g. Flit, C++, Go, Java, Swift, Rust, Haskell, TypeScript, etc.
- Dynamic typing: types are checked at run-time
  - Don't check if an operation is valid until we're actually doing that operation
  - E.g. Python, Ruby, JavaScript, PHP, Racket, Lua
- Recall: implementation language features are *separate* from features of language being implemented

# Dynamic vs. Static Typing

- Static typing: types are checked *before* a program runs
  - E.g. Flit, C++, Go, Java, Swift, Rust, Haskell, TypeScript, etc.
- Dynamic typing: types are checked at run-time
  - Don't check if an operation is valid until we're actually doing that operation
  - E.g. Python, Ruby, JavaScript, PHP, Racket, Lua
- Recall: implementation language features are *separate* from features of language being implemented
- We can implement a dynamically-typed language (Curly-Bool) in a statically typed language (Flit)



```
<expr> ::=  
| <number>  
| <boolean>  
| {+ <expr> <expr>}  
| {* <expr> <expr>}  
| {if <expr> <expr> <expr>}  
| {zero? <expr> }
```

- Allow boolean literals

```
<expr> ::=  
| <number>  
| <boolean>  
| {+ <expr> <expr>}  
| {* <expr> <expr>}  
| {if <expr> <expr> <expr>}  
| {zero? <expr> }
```

- Allow boolean literals
- if instead of if0

```
<expr> ::=  
| <number>  
| <boolean>  
| {+ <expr> <expr>}  
| {* <expr> <expr>}  
| {if <expr> <expr> <expr>}  
| {zero? <expr> }
```

- Allow boolean literals
- if instead of if0
- One operation producing a boolean

# Example Programs

# Example Programs

```
{if {zero? {+ {* 40 50}}  
    {* -1 2000}}  
  1  
 -1}
```

# Representing Booleans

# Representing Booleans

```
(define-type Value  
  (numV [n : Number])  
  (boolV [b : Boolean]))
```

- Datatype

# Representing Booleans

```
(define-type Value  
  (numV [n : Number])  
  (boolV [b : Boolean]))
```

- Datatype
  - Value can be number OR a boolean



# Representing Booleans

```
(define-type Value  
  (numV [n : Number])  
  (boolV [b : Boolean]))
```

- Datatype
  - Value can be number OR a boolean
  - Use pattern-matching to check what a value

# Representing Booleans

```
(define-type Value  
  (numV [n : Number])  
  (boolV [b : Boolean]))
```

- Datatype
  - Value can be number OR a boolean
  - Use pattern-matching to check what a value
- The constructor records run-time type information

- Rename Expression constructors for consistency

- Rename Expression constructors for consistency
  - Add E to names for Expression

- Rename Expression constructors for consistency
  - Add E to names for Expression
  - V was for Value

- Rename Expression constructors for consistency
  - Add E to names for Expression
  - V was for Value

# Abstract Syntax

- Rename Expression constructors for consistency
  - Add E to names for Expression
  - V was for Value

```
(define-type Exp
  (numE [n : Number])
  (boolE [b : Boolean])
  (plusE [left : Exp
           [right : Exp]])
  (timesE [left : Exp
            [right : Exp]])
  (cndE [test : Exp
         [thenCase : Exp]
         [elseCase : Exp]]))
```

## Following the Types

- Interpreter now gives an error



## Following the Types

- Interpreter now gives an error

# Following the Types

- Interpreter now gives an error

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(plusE l r)
     (+ (interp l) (interp r))] ;; TYPE ERROR
```

- (interp l) and (interp r) produce Value

# Following the Types

- Interpreter now gives an error

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(plusE l r)
     (+ (interp l) (interp r))] ;; TYPE ERROR
```

- (interp l) and (interp r) produce Value
- + expects Number

## Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation

## Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:

# Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:
  - Pattern-match on values

# Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:
  - Pattern-match on values
  - If they're both numbers, then do the operation

# Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:
  - Pattern-match on values
  - If they're both numbers, then do the operation
    - and wrap the result in numV to get a Value



# Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:
  - Pattern-match on values
  - If they're both numbers, then do the operation
    - and wrap the result in `numV` to get a `Value`
  - Otherwise, raise an error

# Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:
  - Pattern-match on values
  - If they're both numbers, then do the operation
    - and wrap the result in `numV` to get a `Value`
  - Otherwise, raise an error

# Dynamic Type Checks

- Can define *lifting* function which turns a number-operation into value-operation
- Main idea:
  - Pattern-match on values
  - If they're both numbers, then do the operation
    - and wrap the result in numV to get a Value
  - Otherwise, raise an error

```
(define (lift-binop [op : (Number Number -> Number)]  
          [v1 : Value]  
          [v2 : Value])  
  
  : Value  
  (type-case Value v1  
    [(numV n1)  
     (type-case Value v2  
       [(numV n2)  
        (numV (op n1 n2))]  
       [else (error 'lift-binop "expects RHS to be a number")]])]  
    [else (error 'lift-binop "expects LHS to be a number")]))
```

# Using the Lifting

# Using the Lifting

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(plusE l r)
     (lift-binop + (interp l) (interp r))]
    [(timesE l r)
     (lift-binop * (interp l) (interp r))])
```

- Using a higher-order function means we can use for `plus` and `times`

# Using the Lifting

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(plusE l r)
     (lift-binop + (interp l) (interp r))]
    [(timesE l r)
     (lift-binop * (interp l) (interp r))])
```

- Using a higher-order function means we can use for `plus` and `times`
- Textbook just defines `add` operation that works on values

# Interpreting Booleans

- Boolean literals: no work to do

# Interpreting Booleans

- Boolean literals: no work to do
  - Why return `boolV` and not `boolE` ?



# Interpreting Booleans

- Boolean literals: no work to do
  - Why return `boolV` and not `boolE` ?

# Interpreting Booleans

- Boolean literals: no work to do
  - Why return `boolV` and not `boolE`?

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    [(boolE b)
     (boolV b)]
```

## Dynamic Typing of Conditionals

- Want the thing we're testing to be boolean

# Dynamic Typing of Conditionals

- Want the thing we're testing to be boolean
  - Don't care about the types of the other things

# Dynamic Typing of Conditionals

- Want the thing we're testing to be boolean
  - Don't care about the types of the other things
- Otherwise, same as `if`

## Dynamic Typing of Conditionals

- Want the thing we're testing to be boolean
  - Don't care about the types of the other things
- Otherwise, same as `if`

# Dynamic Typing of Conditionals

- Want the thing we're testing to be boolean
  - Don't care about the types of the other things
- Otherwise, same as `if0`

```
;; Evaluate Expressions
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(cndE test thn els)
     (type-case Value (interp test)
       [(boolV b)
        (if b
            (interp thn)
            (interp els))]
       [else (error 'interp "Non-boolean given to if")]))])
```

## Boolean-Producing Expression

- Booleans not much good unless we can use them



## Boolean-Producing Expression

- Booleans not much good unless we can use them
- Comparison `{zero? x}`

## Boolean-Producing Expression

- Booleans not much good unless we can use them
- Comparison `{zero? x}`

# Boolean-Producing Expression

- Booleans not much good unless we can use them
- Comparison {zero? x}

```
(define (interp [e : Exp] ) : Value
  (type-case Exp e
    ...
    [(zero?E e)
     (type-case Value (interp e)
       [(numV n)
        (boolV (= n 0))]
       [else
        (error 'interp "Expected number")]]])
```

- We have `{zero? #t}` as a type error

# Design Decision

- We have `{zero? #t}` as a type error
- This is not the only choice

- We have `{zero? #t}` as a type error
- This is not the only choice
  - `#t` and `#f` are not zero, so we could produce false

- We have `{zero? #t}` as a type error
- This is not the only choice
  - `#t` and `#f` are not zero, so we could produce false
- Likewise, we could make `if` work on numbers

- We have `{zero? #t}` as a type error
- This is not the only choice
  - `#t` and `#f` are not zero, so we could produce false
- Likewise, we could make `if` work on numbers
  - e.g. in Untyped Racket, `#f` is false and everything else is true



# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

```
{if {zero? 5} #t 10}
```

- Valid Curly-Bool program

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

```
{if {zero? 5} #t 10}
```

- Valid Curly-Bool program
  - Runs without error

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

```
{if {zero? 5} #t 10}
```

- Valid Curly-Bool program
  - Runs without error
- Type isn't checked until we run the branch

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

```
{if {zero? 5} #t 10}
```

- Valid Curly-Bool program
  - Runs without error
- Type isn't checked until we run the branch
  - Doesn't care that `#t` and `10` have different types

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

```
{if {zero? 5} #t 10}
```

- Valid Curly-Bool program
  - Runs without error
- Type isn't checked until we run the branch
  - Doesn't care that #t and 10 have different types
- The same is not true of Flit

# Dynamic Typing of Expressions

- With dynamic types, values have types, but expressions do not

```
{if {zero? 5} #t 10}
```

- Valid Curly-Bool program
  - Runs without error
- Type isn't checked until we run the branch
  - Doesn't care that #t and 10 have different types
- The same is not true of Flit
  - e.g. try it out



# Desugaring

---

# What is syntactic sugar

- Some language features strictly increase the power of a language

# What is syntactic sugar

- Some language features strictly increase the power of a language
  - Let you do things that can't be done any other way

# What is syntactic sugar

- Some language features strictly increase the power of a language
  - Let you do things that can't be done any other way
- Some language features aren't strictly necessary, but are nice to have

# What is syntactic sugar

- Some language features strictly increase the power of a language
  - Let you do things that can't be done any other way
- Some language features aren't strictly necessary, but are nice to have
  - Otherwise we'd just code in machine code/assembly all the time

# What is syntactic sugar

- Some language features strictly increase the power of a language
  - Let you do things that can't be done any other way
- Some language features aren't strictly necessary, but are nice to have
  - Otherwise we'd just code in machine code/assembly all the time
- These “nice to have” are called *syntactic sugar*

# What is syntactic sugar

- Some language features strictly increase the power of a language
  - Let you do things that can't be done any other way
- Some language features aren't strictly necessary, but are nice to have
  - Otherwise we'd just code in machine code/assembly all the time
- These “nice to have” are called *syntactic sugar*
  - They “sweeten” the experience of programming

## Example of Sugar



## Example of Sugar

```
for (int i = 0; i < 10; i++){  
    f();  
}  
// same as  
int i = 0;  
while (i < 10){  
    f();  
    i++;  
}
```

- The C++ compiler doesn't need to know about for-loops

## Example of Sugar

```
for (int i = 0; i < 10; i++){  
    f();  
}  
// same as  
int i = 0;  
while (i < 10){  
    f();  
    i++;  
}
```

- The C++ compiler doesn't need to know about for-loops
- Can do an AST->AST transformation

## Example of Sugar

```
for (int i = 0; i < 10; i++){  
    f();  
}  
// same as  
int i = 0;  
while (i < 10){  
    f();  
    i++;  
}
```

- The C++ compiler doesn't need to know about for-loops
- Can do an AST->AST transformation
  - One less case to handle in the compiler

## Example of Sugar

```
for (int i = 0; i < 10; i++){  
    f();  
}  
// same as  
int i = 0;  
while (i < 10){  
    f();  
    i++;  
}
```

- The C++ compiler doesn't need to know about for-loops
- Can do an AST->AST transformation
  - One less case to handle in the compiler
  - Simplifies the core language

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features
- Desugared features: no case in the interpreter

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features
- Desugared features: no case in the interpreter
  - Instead, translate to a smaller “core” AST type



- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features
- Desugared features: no case in the interpreter
  - Instead, translate to a smaller “core” AST type
  - Keeps the interpreter small, easier to maintain

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features
- Desugared features: no case in the interpreter
  - Instead, translate to a smaller “core” AST type
  - Keeps the interpreter small, easier to maintain
- Have two types for AST

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features
- Desugared features: no case in the interpreter
  - Instead, translate to a smaller “core” AST type
  - Keeps the interpreter small, easier to maintain
- Have two types for AST
  - Surface AST

- When one feature can be expressed in terms of another, sometimes we implement it by *desugaring*
  - Translating the AST for a feature into other language features
- Desugared features: no case in the interpreter
  - Instead, translate to a smaller “core” AST type
  - Keeps the interpreter small, easier to maintain
- Have two types for AST
  - Surface AST
  - Core AST

## Example: Subtraction

- `{- <expr> <expr>}`

## Example: Subtraction

- `{- <expr> <expr>}`
- We'll add subtraction to our language

## Example: Subtraction

- $\{- \text{ <expr> } \text{ <expr> }\}$
- We'll add subtraction to our language
  - *without changing the interpreter at all*

## Example: Subtraction

- `{- <expr> <expr>}`
- We'll add subtraction to our language
  - *without changing the interpreter at all*
- Separate AST into surface and core AST



## Example: Subtraction

- `{- <expr> <expr>}`
- We'll add subtraction to our language
  - *without changing the interpreter at all*
- Separate AST into surface and core AST
- Add translation from surface to core AST

## Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types

# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”

# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”
    - S for “surface”, or “sugar”

# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”
    - S for “surface”, or “sugar”
    - Corresponds directly to the source code

# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”
    - S for “surface”, or “sugar”
    - Corresponds directly to the source code
    - Program before desugaring

# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”
    - S for “surface”, or “sugar”
    - Corresponds directly to the source code
    - Program before desugaring
  - Exp for core expressions

# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”
    - S for “surface”, or “sugar”
    - Corresponds directly to the source code
    - Program before desugaring
  - Exp for core expressions
    - The result of desugaring



# Different Syntax Tree Types

- We will define two separate Abstract Syntax Tree types
  - ExpS for “Surface expressions”
    - S for “surface”, or “sugar”
    - Corresponds directly to the source code
    - Program before desugaring
  - Exp for core expressions
    - The result of desugaring
    - The type of tree we give to `interp` to actually run

## Subtraction: Datatype

- First need the surface AST

## Subtraction: Datatype

- First need the surface AST

## Subtraction: Datatype

- First need the surface AST

```
(define-type ExpS
  (numS [n : Number])
  (plusS [left : ExpS]
         [right : ExpS])
  (timesS [left : ExpS]
          [right : ExpS])
  (cndS [test : ExpS]
        [thenBranch : ExpS]
        [elseBranch : ExpS])
  (subS [left : ExpS]
        [right : ExpS]))
```

- Exact same as core AST, but with subtraction

## Subtraction: Parser Case

- Parser now produces a surface expression

## Subtraction: Parser Case

- Parser now produces a surface expression
  - We'll add another pass to make a core expression

## Subtraction: Parser Case

- Parser now produces a surface expression
  - We'll add another pass to make a core expression

## Subtraction: Parser Case

- Parser now produces a surface expression
  - We'll add another pass to make a core expression

```
(define (parse [s : S-Exp]) : ExpS
  ...
  [(s-exp-match? `{- ANY ANY} s)
   (subS (parse (second (s-exp->list s)))
          (parse (third (s-exp->list s))))])
```



## Desugaring: Core Cases

- Define function that translates from  $\text{ExpS}$  to  $\text{Exp}$

## Desugaring: Core Cases

- Define function that translates from  $\text{ExpS}$  to  $\text{Exp}$

# Desugaring: Core Cases

- Define function that translates from `ExpS` to `Exp`

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    [(numS n)
     (numE n)]
    [(boolS b)
     (boolE b)]
    [(plusS l r)
     (plusE (desugar l) (desugar r))]
    [(timesS l r)
     (timesE (desugar l) (desugar r))]
    ...
```

- For features that are in the core

# Desugaring: Core Cases

- Define function that translates from ExpS to Exp

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    [(numS n)
     (numE n)]
    [(boolS b)
     (boolE b)]
    [(plusS l r)
     (plusE (desugar l) (desugar r))]
    [(timesS l r)
     (timesE (desugar l) (desugar r))]
    ...
```

- For features that are in the core
  - Pattern match

# Desugaring: Core Cases

- Define function that translates from ExpS to Exp

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    [(numS n)
     (numE n)]
    [(boolS b)
     (boolE b)]
    [(plusS l r)
     (plusE (desugar l) (desugar r))]
    [(timesS l r)
     (timesE (desugar l) (desugar r))]
    ...
```

- For features that are in the core
  - Pattern match
  - Recursively desugar the parts

# Desugaring: Core Cases

- Define function that translates from ExpS to Exp

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    [(numS n)
     (numE n)]
    [(boolS b)
     (boolE b)]
    [(plusS l r)
     (plusE (desugar l) (desugar r))]
    [(timesS l r)
     (timesE (desugar l) (desugar r))]
    ...
```

- For features that are in the core
  - Pattern match
  - Recursively desugar the parts
  - Re-assemble with the corresponding Exp constructor

## Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*

## Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:



## Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:
  - make a new syntax tree

# Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:
  - make a new syntax tree
  - that evaluates to the result of the desired operation

# Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:
  - make a new syntax tree
  - that evaluates to the result of the desired operation
  - without

# Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:
  - make a new syntax tree
  - that evaluates to the result of the desired operation
  - without
    - running the code

# Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:
  - make a new syntax tree
  - that evaluates to the result of the desired operation
  - without
    - running the code
    - inspecting the trees

# Desugaring: Subtraction Helper Function

- Desugaring is about *source-to-source transformations*
- Given some syntax trees, how can we:
  - make a new syntax tree
  - that evaluates to the result of the desired operation
  - without
    - running the code
    - inspecting the trees
- Example:

## Desugaring: Subtraction Helper Function

- There is no `subE` in the `Exp` definition

## Desugaring: Subtraction Helper Function

- There is no `subE` in the `Exp` definition
  - Could add it, but don't need to



## Desugaring: Subtraction Helper Function

- There is no subE in the Exp definition
  - Could add it, but don't need to
  - Instead, translate it into *existing* features of Exp

## Desugaring: Subtraction Helper Function

- There is no `subE` in the `Exp` definition
  - Could add it, but don't need to
  - Instead, translate it into *existing* features of `Exp`
    - e.g. Our *core syntax*

## Desugaring: Subtraction Helper Function

- There is no `subE` in the `Exp` definition
  - Could add it, but don't need to
  - Instead, translate it into *existing* features of `Exp`
    - e.g. Our *core syntax*

## Desugaring: Subtraction Helper Function

- There is no `subE` in the `Exp` definition
  - Could add it, but don't need to
  - Instead, translate it into *existing* features of `Exp`
    - e.g. Our *core syntax*

```
(define (subE [l : Exp]
              [r : Exp])
  : Exp
  (plusE l
        (timesE (numE -1) r)))
```

- e.g.  $\{-\ x\ y\}$  becomes  $\{+\ x\ \{\ast\ -1\ y\}\}$

## Desugaring: Subtraction Helper Function

- There is no `subE` in the `Exp` definition
  - Could add it, but don't need to
  - Instead, translate it into *existing* features of `Exp`
    - e.g. Our *core syntax*

```
(define (subE [l : Exp]
              [r : Exp])
  : Exp
  (plusE l
        (timesE (numE -1) r)))
```

- e.g.  $\{- x y\}$  becomes  $\{+ x \{* -1 y\}\}$
- Note: `subE` is a function, *not* a constructor

## Desugaring: Subtraction

- Can use our helper function in desugar now

## Desugaring: Subtraction

- Can use our helper function in desugar now

# Desugaring: Subtraction

- Can use our helper function in desugar now

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    ...
    [(subS l r)
     (subE (desugar l) (desugar r))])
```

- Why did we desugar `l` and `r`?



# Desugaring: Subtraction

- Can use our helper function in desugar now

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    ...
    [(subS l r)
     (subE (desugar l) (desugar r))])
```

- Why did we desugar `l` and `r`?
  - Can't give to `plusE` until they're expressions

# Desugaring: Subtraction

- Can use our helper function in desugar now

```
(define (desugar [es : ExpS]) : Exp
  (type-case ExpS es
    ...
    [(subS l r)
     (subE (desugar l) (desugar r))])
```

- Why did we desugar `l` and `r`?
  - Can't give to `plusE` until they're expressions
  - e.g. what if `-` occurs deep in an expression

## More Examples: Equality

- $\{= x y\}$

## More Examples: Equality

- $\{= \ x \ y\}$ 
  - produces a boolean for if they're equal

## More Examples: Equality

- $\{= \ x \ y\}$ 
  - produces a boolean for if they're equal
- Can we implement a boolean-returning equality check

## More Examples: Equality

- $\{= \ x \ y\}$ 
  - produces a boolean for if they're equal
- Can we implement a boolean-returning equality check
  - Using just what we have now?

## More Examples: Equality

- $\{= x y\}$ 
  - produces a boolean for if they're equal
- Can we implement a boolean-returning equality check
  - Using just what we have now?
- $x = y$  if and only if  $x - y = 0$

## More Examples: Equality

- $\{= x\ y\}$ 
  - produces a boolean for if they're equal
- Can we implement a boolean-returning equality check
  - Using just what we have now?
- $x = y$  if and only if  $x - y = 0$



## More Examples: Equality

- $\{ = \ x \ y \}$ 
  - produces a boolean for if they're equal
- Can we implement a boolean-returning equality check
  - Using just what we have now?
- $x = y$  if and only if  $x - y = 0$

```
(define-type ExpS
  ...
  (eqS [l : ExpS] [r : ExpS]))

(define (eqE [l : Exp]
             [r : Exp])
  : Exp
  (zero?E (subE l r)))

(define (desugar [es : ExpS]) : Exp
  ...
  [(eqS l r)
   (eqE (desugar l) (desugar r))])
```

## More Examples: Boolean Operations

- Can we add and, or, and not?

## More Examples: Boolean Operations

- Can we add and, or, and not?
- E.g. and

## More Examples: Boolean Operations

- Can we add and, or, and not?
- E.g. and
  - Want `a and b` to be `True`, false otherwise

## More Examples: Boolean Operations

- Can we add and, or, and not?
- E.g. and
  - Want `and #t #t` to be `#t`, false otherwise
  - Notice: if first arg is `#t`, then second arg is the result of the whole and

## More Examples: Boolean Operations

- Can we add and, or, and not?
- E.g. and
  - Want `and #t #t` to be `#t`, false otherwise
  - Notice: if first arg is `#t`, then second arg is the result of the whole and

## More Examples: Boolean Operations

- Can we add and, or, and not?
- E.g. and
  - Want and #t #t to be #t, false otherwise
  - Notice: if first arg is #t, then second arg is the result of the whole and

```
(define (andE [l : Exp]
              [r : Exp])
  : Exp
  (cndE l
        r
        (boolE #f)))
```

```
(define (orE [l : Exp]
             [r : Exp])
  (cndE l (boolE #t) r))
```

```
(define (notE [e : Exp])
  (cndE e (boolE #f) boolE #t))
```

# Advantages of Desugaring

- We added subtraction to Curly



# Advantages of Desugaring

- We added subtraction to Curly
  - *without* changing Exp or interp

# Advantages of Desugaring

- We added subtraction to Curly
  - *without* changing Exp or interp
- Avoids repetition of code

# Advantages of Desugaring

- We added subtraction to Curly
  - *without* changing `Exp` or `interp`
- Avoids repetition of code
  - Any + bug-fixes/optimizations now also affect -

# Advantages of Desugaring

- We added subtraction to Curly
  - *without* changing `Exp` or `interp`
- Avoids repetition of code
  - Any + bug-fixes/optimizations now also affect -
- Simplifies the interpreter

# Advantages of Desugaring

- We added subtraction to Curly
  - *without* changing `Exp` or `interp`
- Avoids repetition of code
  - Any + bug-fixes/optimizations now also affect -
- Simplifies the interpreter
  - `interp` is usually the most complicated part

# Compilation vs. Desugaring

- Technically we just wrote our first compiler

- Technically we just wrote our first compiler
  - Translated a small language into an even smaller one

# Compilation vs. Desugaring

- Technically we just wrote our first compiler
  - Translated a small language into an even smaller one
- Compilation is just a bunch of desugaring passes



# Compilation vs. Desugaring

- Technically we just wrote our first compiler
  - Translated a small language into an even smaller one
- Compilation is just a bunch of desugaring passes
  - Simpler and simpler languages until we have something simple enough for assembly code

## Example

- $\{\text{if } \{= \{+ 3 3\} 3\} 4 5\}$

## Example

- $\{\text{if } \{= \{+ 3 3\} 3\} 4 5\}$
- First we desugar

## Example

- `{if {= {+ 3 3} 3} 4 5}`
- First we desugar
  - `{if {zero? {- {+ 3 3} 3}} 4 5}`

## Example

- `{if {= {+ 3 3} 3} 4 5}`
- First we desugar
  - `{if {zero? {- {+ 3 3} 3}} 4 5}`
  - `{if {zero? {+ {+ 3 3} {* -1 3}} 4 5}`

## Example

- `{if {= {+ 3 3} 3} 4 5}`
- First we desugar
  - `{if {zero? {- {+ 3 3} 3}} 4 5}`
  - `{if {zero? {+ {+ 3 3} {* -1 3}}} 4 5}`
  - Actual desugaring happens on trees, not on concrete syntax

## Example

- `{if {= {+ 3 3} 3} 4 5}`
- First we desugar
  - `{if {zero? {- {+ 3 3} 3}} 4 5}`
  - `{if {zero? {+ {+ 3 3} {* -1 3}}} 4 5}`
  - Actual desugaring happens on trees, not on concrete syntax
  - But this is more readable

## Example

- `{if {= {+ 3 3} 3} 4 5}`
- First we desugar
  - `{if {zero? {- {+ 3 3} 3}} 4 5}`
  - `{if {zero? {+ {+ 3 3} {* -1 3}}} 4 5}`
  - Actual desugaring happens on trees, not on concrete syntax
  - But this is more readable
- Then we interpret



- Then we interpret

- Then we interpret
  - Dynamic type checks left off for readability

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 {* -1 3}}}} 4 5)`

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 -3}} 4 5)`

## Example ctd

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 -3}} 4 5)`
  - `(interp {if {zero? 3} 4 5)`

## Example ctd

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 -3}} 4 5)`
  - `(interp {if {zero? 3} 4 5)`
  - `(interp {if #f 4 5})`

## Example ctd

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 -3}} 4 5)`
  - `(interp {if {zero? 3} 4 5)`
  - `(interp {if #f 4 5})`
  - `(interp 5)`



## Example ctd

- Then we interpret
  - Dynamic type checks left off for readability
  - `(interp {if {zero? {+ {+ 3 3} {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 {* -1 3}}}} 4 5)`
  - `(interp {if {zero? {+ 6 -3}} 4 5)`
  - `(interp {if {zero? 3} 4 5)`
  - `(interp {if #f 4 5})`
  - `(interp 5)`
  - 5